COM232 VISUAL PROGRAMMING FOR SOFTWARE DESIGN

“Car Rental Management System”

Mohammed Alhamss.

B2205.010024.

Overview:

I have developed a Car Rental Management System designed to streamline the car rental process. Users can select a car from a table, and the system records the rental details, including the car's manufacturing year. Based on that information, the rental price is determined.

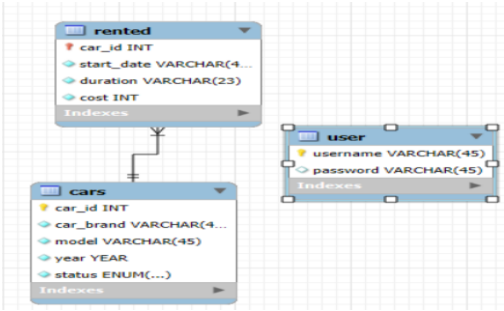
The system also includes an admin section, where administrators can:

- Review of rental sales.
- Add new cars to the database.

The system was built using Java for application logic and MySQL for database management.

Database:

Three tables were created in the database: rented, cars, and user.



System Interface Overview:

Below is a visual representation of the system interface, along with a description of each image:

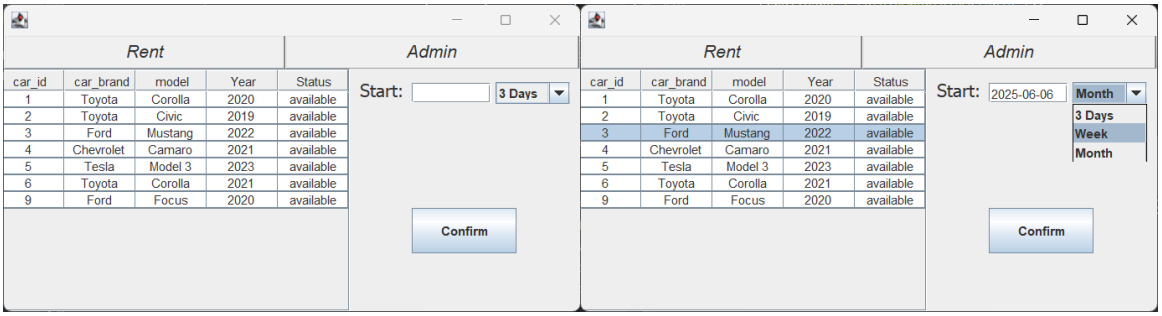


Figure 1: Main menu allowing users to select a car and set rental date and duration.

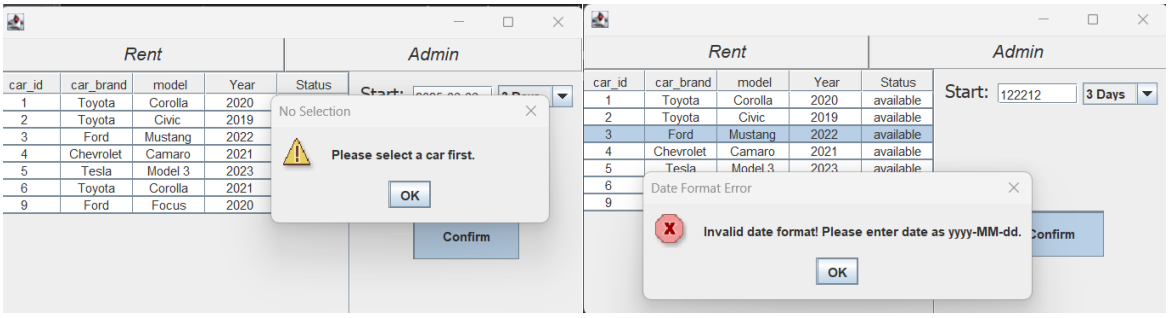


Figure 2: Handling user input errors or cases where no item is selected.

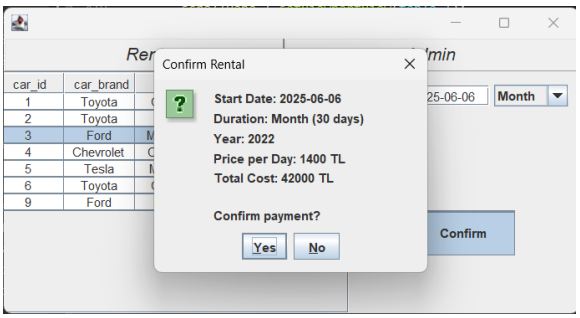


Figure 3: A JOptionPane dialog showing confirmation details including start date, rental duration, car year, price per day, and total cost.

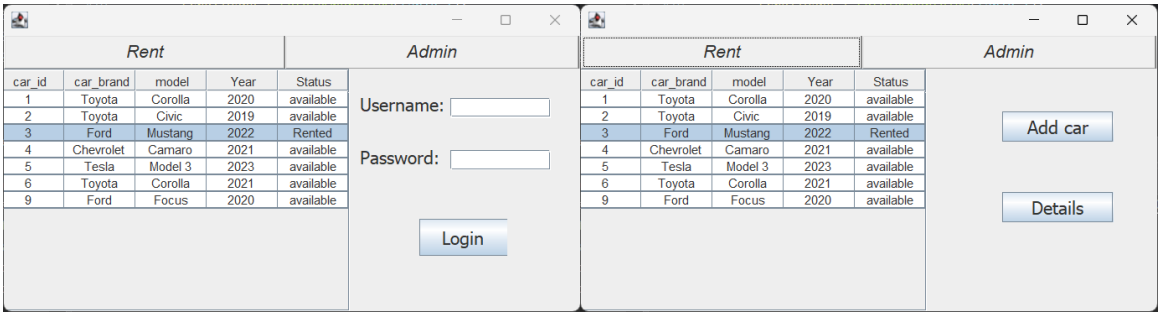


Figure 4: User login screen with an option for admin access, allowing car additions and viewing of sales details.

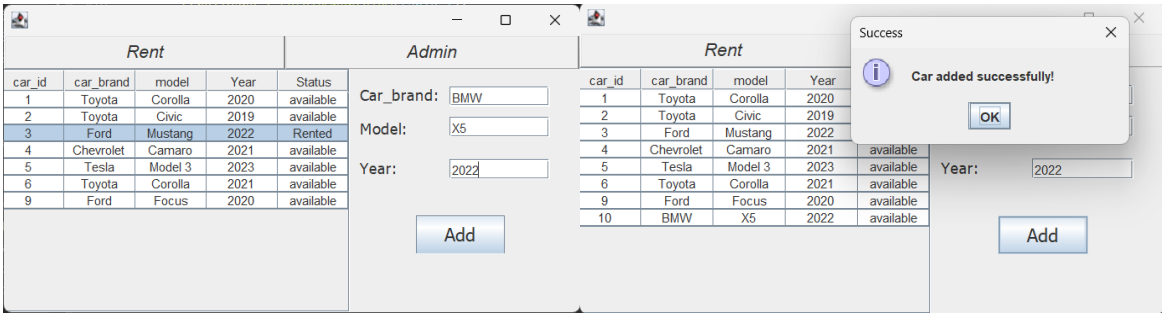


Figure 5: Process of adding a new car to the system.

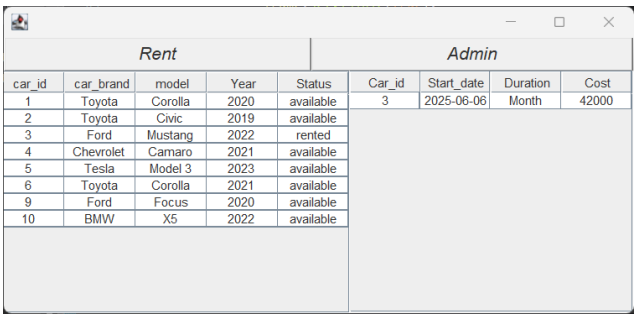


Figure 6: Sales details overview.

Source Code Explanation:

```java
    public void actionPerformed(ActionEvent e) {
        int selectedRow = table.getSelectedRow();
        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(workframe.this, "Please select a car first.", "No Selection",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }
```

```java
String yearStr = table.getValueAt(selectedRow, 3).toString();
int year = Integer.parseInt(yearStr);

String startDate = start_date.getText().trim();


SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
sdf.setLenient(false);

try {
    sdf.parse(startDate);
} catch (ParseException pe) {
    JOptionPane.showMessageDialog(workframe.this,
            "Invalid date format! Please enter date as yyyy-MM-dd.",
            "Date Format Error", JOptionPane.ERROR_MESSAGE);
    return;
}

String durationStr = comboBox.getSelectedItem().toString();

int days = switch (durationStr) {
case "3 Days" -> 3;
case "Week" -> 7;
case "Month" -> 30;
default -> 0;
};
```

```java
int carId = Integer.parseInt(table.getValueAt(selectedRow, 0).toString());
man.rentCar(carId, startDate, durationStr, totalCost);
model.setValueAt("Rented", selectedRow, 4);
model_1.setRowCount(0);
fillTable_1(man.getDetails());
JOptionPane.showMessageDialog(workframe.this, "Car rented successfully!", "Success",
        JOptionPane.INFORMATION_MESSAGE);
```

Figure 7: Code logic for handling car rental — checks if a car is selected, retrieves the car's year and start date, sets the rental price based on the year, retrieves the car ID, and calls the rentCar(**Figure 10**) method to save the rental information to the database.

```java
JButton btnNewButton_1 = new JButton("Login");
btnNewButton_1.setFocusable(false);
btnNewButton_1.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        String user = username.getText();
        String password = String.valueOf(passwordField.getPassword());

        if (user.isEmpty() || password.isEmpty()) {
            JOptionPane.showMessageDialog(panel_1, "Username or password cannot be empty", "Error",
                    JOptionPane.ERROR_MESSAGE);
            return;
        }
        if (man.logCheck(user, password)) {
            JOptionPane.showMessageDialog(panel_1, "Login successful!", "Success",
                    JOptionPane.INFORMATION_MESSAGE);

            cardLayout.show(layeredPane, "info");

        } else {
            JOptionPane.showMessageDialog(panel_1, "Invalid credentials", "Login Failed",
                    JOptionPane.ERROR_MESSAGE);
        }
        username.setText("");
        passwordField.setText("");
```

Figure 8: Login functionality — retrieves and validates username and password input, checks credentials using the logCheck (**Figure 11**)method, displays appropriate messages, and navigates to the next panel upon successful login.

```java
ublic ArrayList<Information> getCars() throws SQLException {

    ArrayList<Information> car = new ArrayList<Information>();
    Statement st = getConnected().createStatement();
    ResultSet rs = st.executeQuery("select* from cars");

    while (rs.next()) {
        Information info = new Information();
        info.setCar_id(rs.getInt(1));
        info.setCar_brand(rs.getString(2));
        info.setModel(rs.getString(3));
        info.setYear(rs.getInt(4));
        info.setStstus(rs.getString(5));
        car.add(info);
```

Figure 9: Method getCars() — retrieves all car records from the cars table in the database, stores each row in an Information object, and returns an ArrayList of cars.

```java
public void rentCar(int carId, String startDate, String duration, int totalCost) throws SQLException {
    Connection con = getConnected();
    String sql = "INSERT INTO rented (car_id, start_date, duration, cost) VALUES (?, ?, ?, ?)";
    PreparedStatement ps = con.prepareStatement(sql);
    ps.setInt(1, carId);
    ps.setString(2, startDate);
    ps.setString(3, duration);
    ps.setInt(4, totalCost);
    ps.executeUpdate();
```

Figure 10: Method rentCar() — inserts rental information into the rented table and updates the car's status to 'Rented' in the cars table using prepared SQL statements.

```java
    Connection con = getConnected();
    String sql = "SELECT * FROM user WHERE username = ? AND password = ?";
    PreparedStatement ps = con.prepareStatement(sql);
    ps.setString(1, username);
    ps.setString(2, password);

    ResultSet rs = ps.executeQuery();
    isValid = rs.next();
    rs.close();
    ps.close();
    con.close();
```

Figure 11: Method logCheck() — verifies user credentials by querying the user table using a prepared statement; returns true if valid credentials are found, otherwise returns false.

```java
public void add_new_car(String brand, String mode_l, int year_) throws SQLException {
    Connection con = getConnected();
    String sql = "INSERT INTO cars ( car_brand,model,year,status) VALUES (?, ?, ?, ?)";
    PreparedStatement ps = con.prepareStatement(sql);

    ps.setString(1, brand);
    ps.setString(2, mode_l);
    ps.setInt(3, year_);
    ps.setString(4, "available");
    ps.executeUpdate();
```

Figure 12: Method add_new_car() — inserts a new car into the cars table with brand, model, and year details; sets the initial status to "available".

Future Plans:

- User Registration System.
- Payment Integration.
- Car Return and Availability Update.
- Responsive UI and Mobile Support.
- Notification System.
- Vehicle Health Monitoring.
- Geofencing Alerts.